Time-based Measurement

Today Agenda

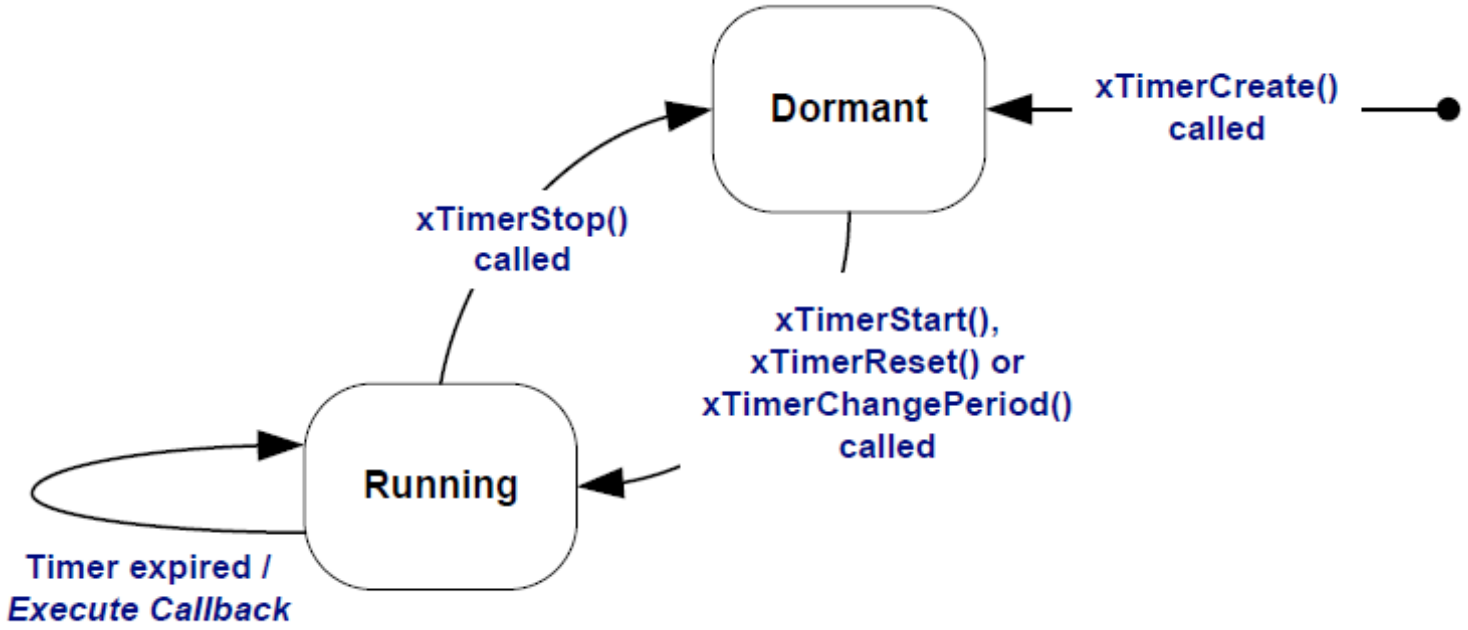
- Software Timer
- AXI Timer
- Timer-based Measurement Example

Required Reading

- *Zynq Book Tutorial.pdf: slides 90-111*
- *FreeRTOS Kernel Tutorial: Chapter 5 (slides 195-255)*

FreeRTOS Software Timers

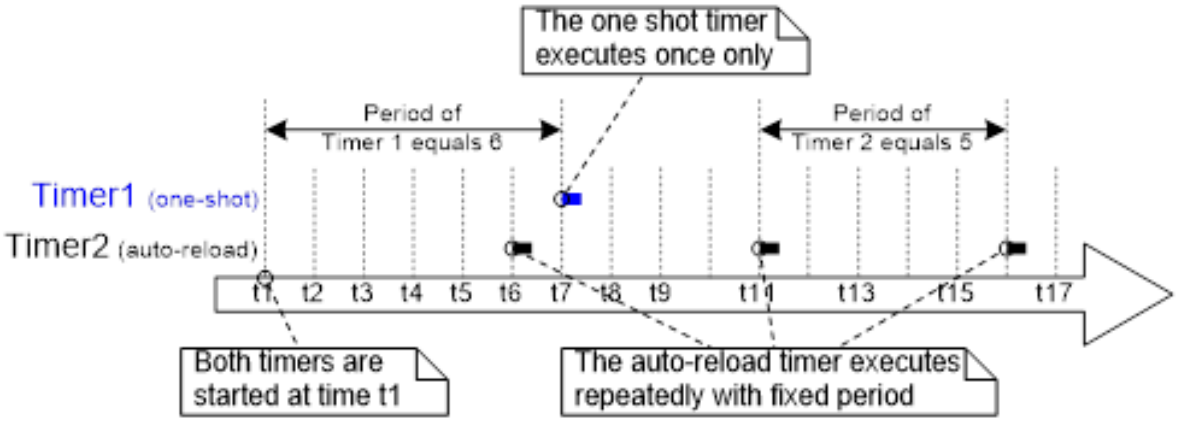
Software timers are used to schedule the execution of a function at a set time in the future or periodically with a fixed frequency. The function executed by the software timer is called the *callback function*.



FreeRTOS Software Timers

Software timers are implemented by and are under the control of the FreeRTOS kernel and do not require hardware support. They are not hardware timers or hardware counters.

Software timers do not use any processing time unless a software timer callback function is actually executing.



FreeRTOS Software Timer Callback Function

Software timer *callback* functions are implemented as C functions. The unique aspect is their prototype, which must return void, and take a *handle* to a software timer as its only parameter.

```
void ATimerCallback( TimerHandle_t xTimer );
```

Software timer callback functions execute from start to finish and exit in the normal way. They should be kept short and must not enter the *Blocked* state.

FreeRTOS Attributes and States of a Software Timer

Period of a software timer is the time between the software timer being started and the software timer's callback function executing.

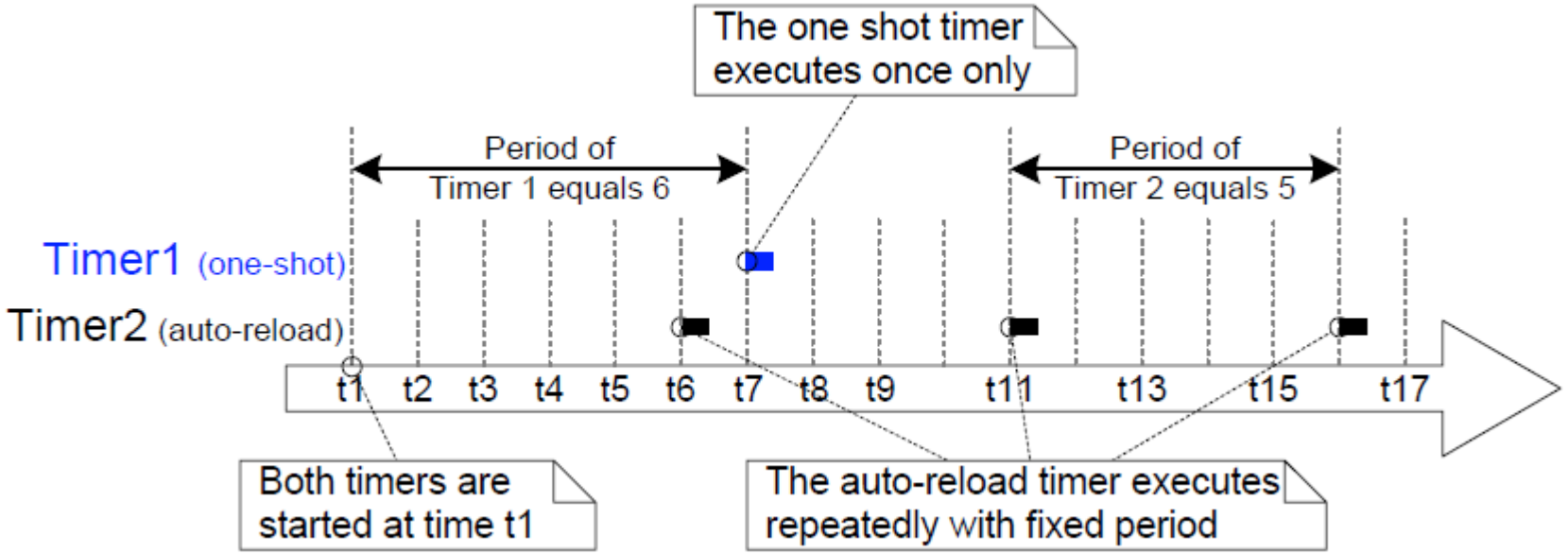
There are two types of software timer:

1. *One-shot timers* once started, will execute its callback function once only. A one-shot timer can be restarted manually, but will not restart itself.

2. *Auto-reload timers* once started, will re-start itself each time it expires, resulting in a periodic execution of its callback function.

FreeRTOS Attributes and States of a Software Timer

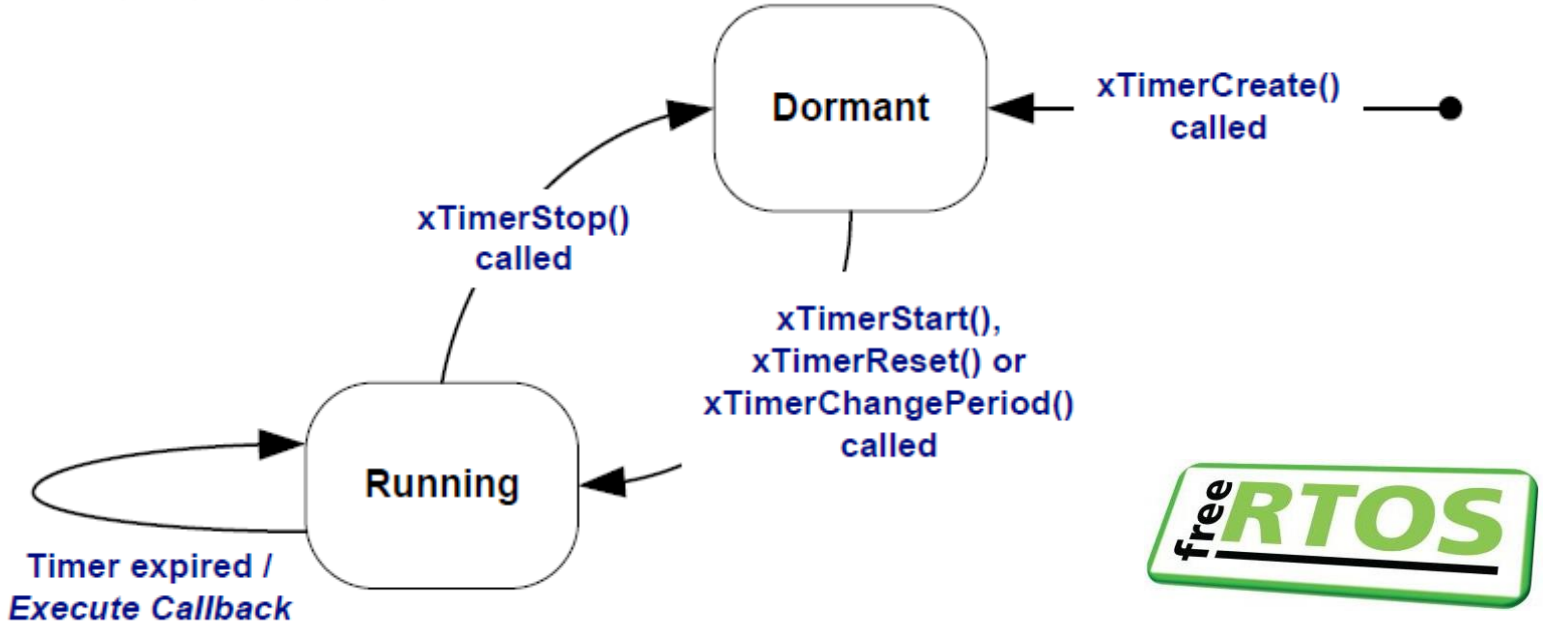
Here is the difference in behavior between a one-shot timer and an auto-reload timer where the dashed vertical lines mark the times at which a *tick* interrupt occurs.



FreeRTOS Attributes and States of a Software Timer

A software timer can be in one of the following two states:

1. A *Dormant* software timer exists and can be referenced by its handle, but is not running, so its callback functions will not execute.



FreeRTOS Attributes and States of a Software Timer

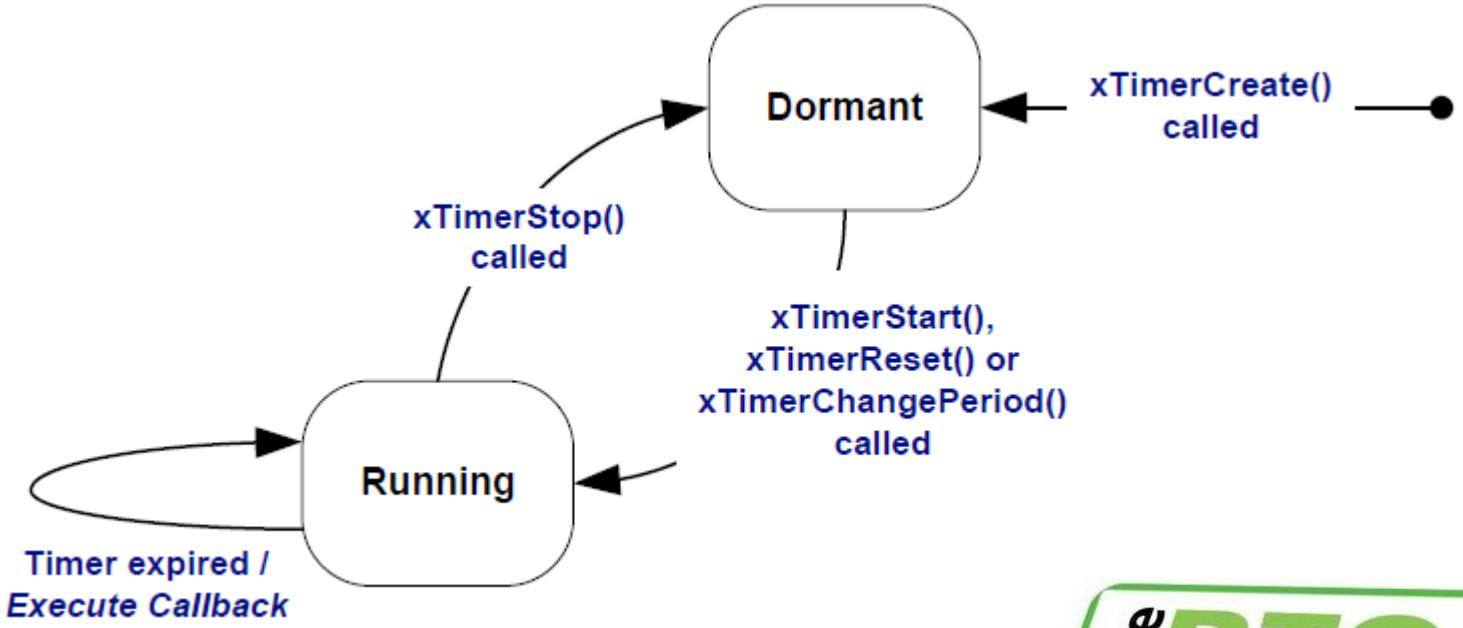
A software timer can be in one of the following two states:

2. A *Running* software timer will execute its callback function after a time equal to its period has elapsed since the software timer entered the *Running* state, or since the software timer was last reset.

The *xTimerDelete()* API function deletes a timer. A timer can be deleted at any time.

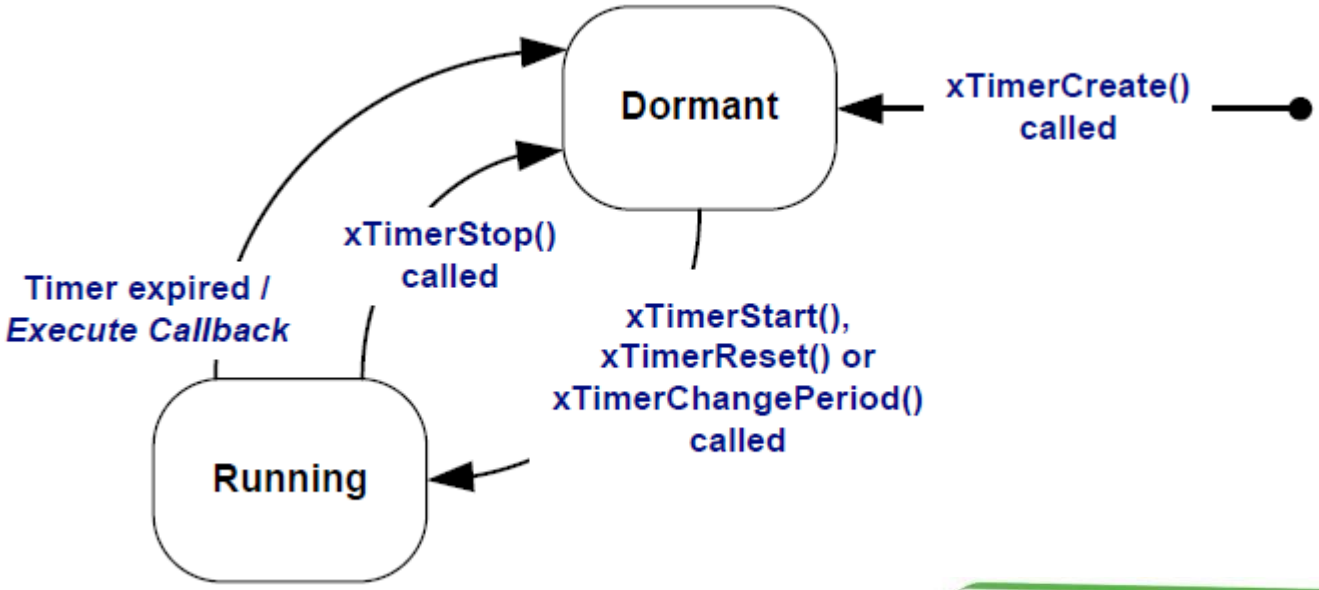
FreeRTOS Attributes and States of a Software Timer

The auto-reload timer executes its callback function then re-enters the *Running* state.



FreeRTOS Attributes and States of a Software Timer

The one-shot timer executes its callback function then enters the *Dormant* state.



FreeRTOS Context of a Software Timer

Software timer functions send commands from the calling task to the daemon task (i.e. background task that does all the timer bookkeeping) on a queue called the *timer command queue*.

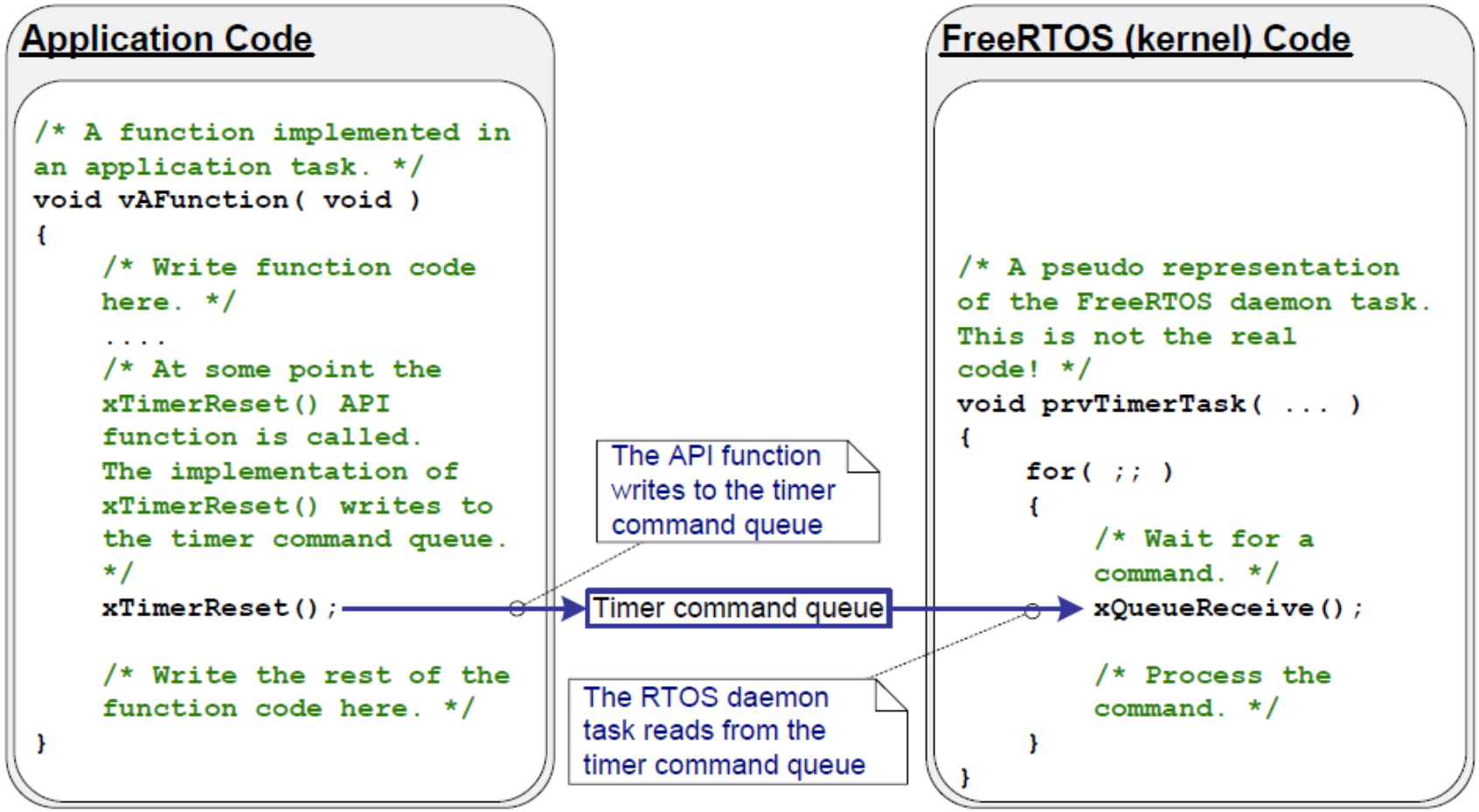
Examples of commands include *start timer*, *stop timer* and *reset timer*. The timer command queue is a standard FreeRTOS queue that is created automatically when the scheduler is started.

FreeRTOS Context of a Software Timer

The length of the timer command queue is set by the *configTIMER_QUEUE_LENGTH* constant in *FreeRTOSConfig.h*.

```
#define configUSE_TIMERS                        1
#define configTIMER_TASK_PRIORITY               (configMAX_PRIORITIES - 1 )
#define configTIMER_QUEUE_LENGTH                20
#define configTIMER_TASK_STACK_DEPTH
                                        ( configMINIMAL_STACK_SIZE * 2 )
```

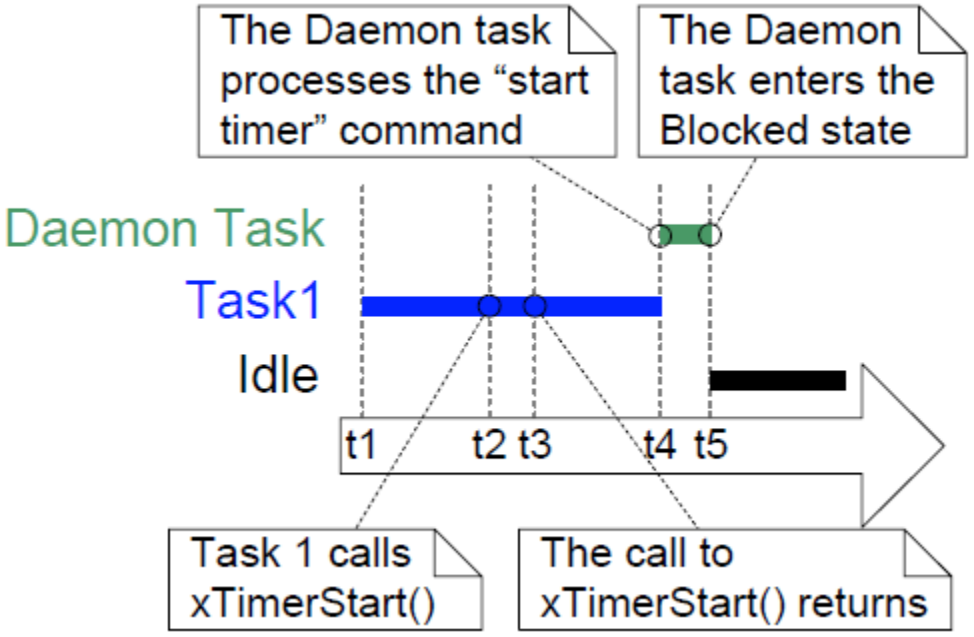
FreeRTOS Context of a Software Timer



FreeRTOS Context of a Software Timer

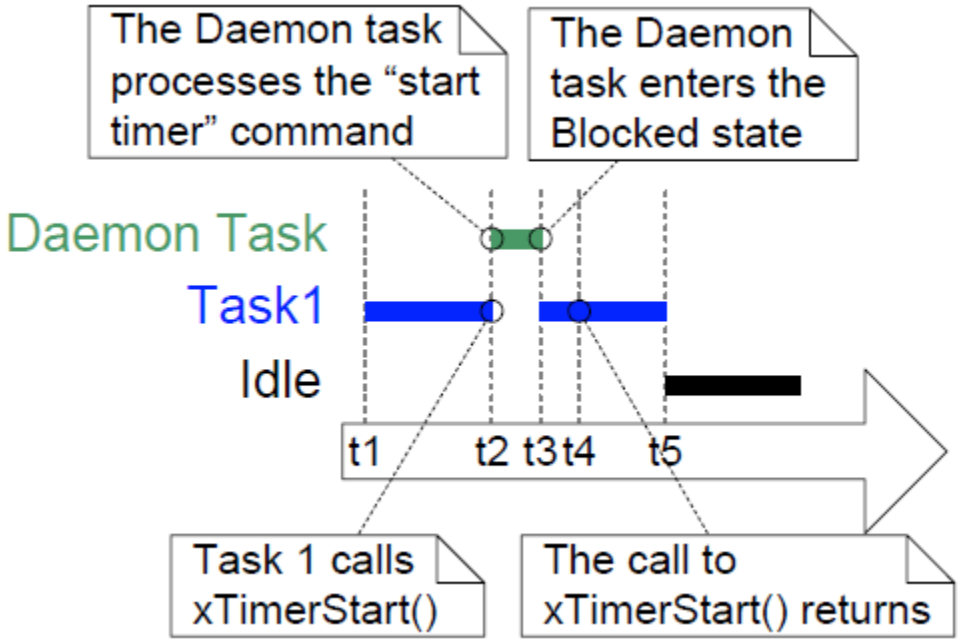
The daemon task is scheduled like any other FreeRTOS task and only processes commands or executes timer callback functions when it is the highest priority task that is able to run.

Here is the execution when the priority of the daemon task is *below* the priority of a task That calls the *xTimerStart()* API function.



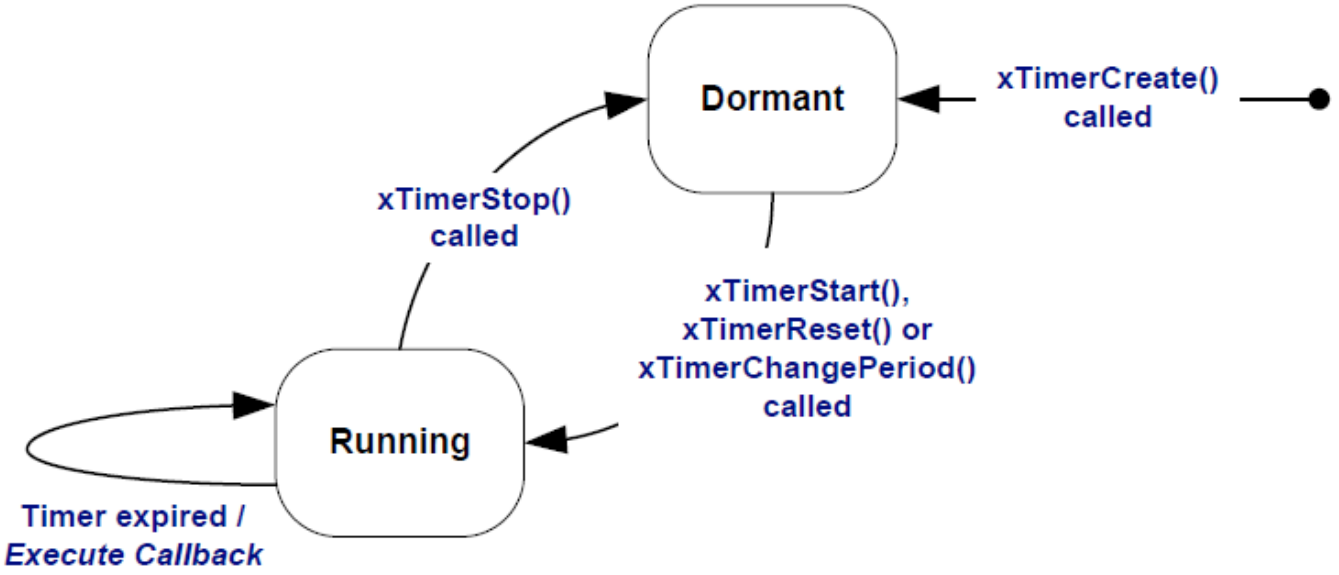
FreeRTOS Context of a Software Timer

Here is a similar scenario but this time the priority of the daemon task is *above* the priority of the task that calls the *xTimerStart()* API function.



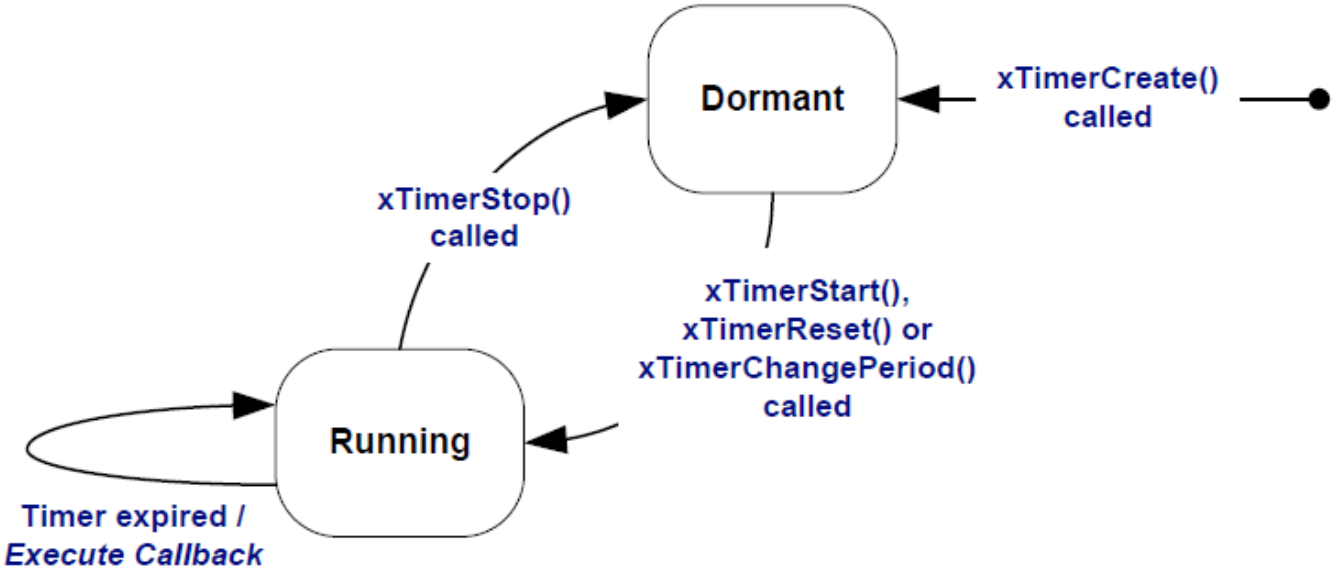
FreeRTOS Context of a Software Timer

Commands sent to the timer command queue contain a time stamp. The time stamp is used to account for any time that passes between a command being sent by an application task and the same command being processed by the daemon task.



FreeRTOS Context of a Software Timer

For example, if a *start timer* command is sent to start a timer that has a period of 10 ticks, the time stamp is used to ensure the timer being started expires 10 ticks after the command was sent, not 10 ticks after the command was processed by the daemon task.



FreeRTOS Software Timer Creation

Software timers are referenced by variables of type TimerHandle_t. *xTimerCreate()* is used to create a software timer and returns a TimerHandle_t to reference the software timer it creates.

Software timers are created in the *Dormant* state. Software timers can be created before the scheduler is running or from a task after the scheduler has been started.

```
TimerHandle_t xTimerCreate( const char * const pcTimerName,
                            TickType_t xTimerPeriodInTicks,
                            UBaseType_t uxAutoReload,
                            void * pvTimerID,
                            TimerCallbackFunction_t pxCallbackFunction );
```

FreeRTOS Software Timer Creation

pcTimerName is a descriptive name for the timer. This is not used by FreeRTOS in any way and is included purely as a debugging aid.

xTimerPeriodInTicks is the period of the timer specified in ticks. The *pdMS_TO_TICKS()* macro is used to convert a time specified in milliseconds into a time specified in ticks.

```
TimerHandle_t xTimerCreate( const char * const pcTimerName,
                            TickType_t xTimerPeriodInTicks,
                            UBaseType_t uxAutoReload,
                            void * pvTimerID,
                            TimerCallbackFunction_t pxCallbackFunction );
```

FreeRTOS Software Timer Creation

*uxAutoReload s*et to *pdTRUE* creates an auto-reload timer and to pdFALSE creates a one-shot timer.

pvTimerID is the ID value of the software timer. The ID is a void pointer, and can be used by the application for any purpose. The ID is particularly useful when the same callback function is used by more than one software timer. *pvTimerID* sets an initial value for the ID of the task being created

```
TimerHandle_t xTimerCreate( const char * const pcTimerName,
                            TickType_t xTimerPeriodInTicks,
                            UBaseType_t uxAutoReload,
                            void * pvTimerID,
                            TimerCallbackFunction_t pxCallbackFunction );
```

FreeRTOS Software Timer Creation

pxCallbackFunction parameter is a pointer to the function to use as the callback function for the software timer being created.

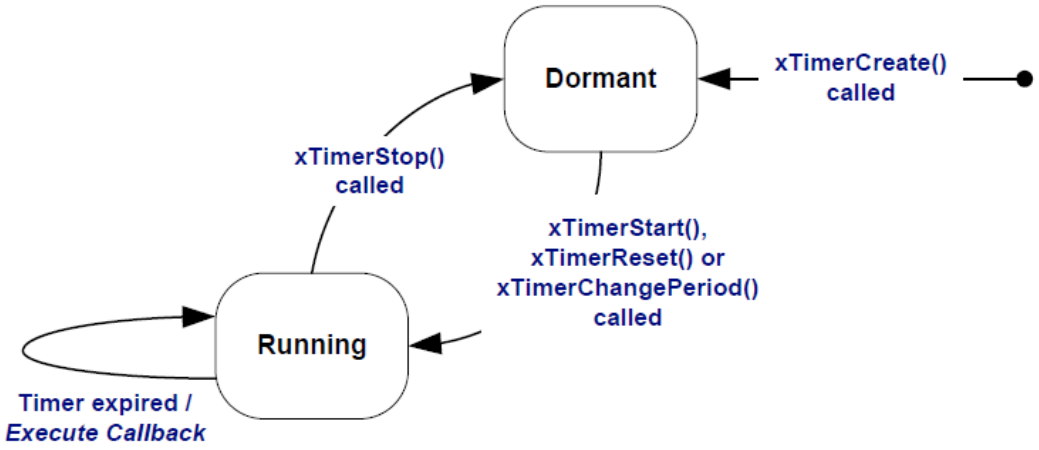
If the returned value is NULL then the software timer cannot be created. A non-NULL value is the handle of the created timer and indicates that the software timer has been created successfully.

```
TimerHandle_t xTimerCreate( const char * const pcTimerName,
                            TickType_t xTimerPeriodInTicks,
                            UBaseType_t uxAutoReload,
                            void * pvTimerID,
                            TimerCallbackFunction_t pxCallbackFunction );
```

FreeRTOS Software Timer Start

xTimerStart() is used to start a software timer that is in the *Dormant* state, or restart a software timer that is in the *Running* state. *xTimerStart()* can be called before the scheduler is started, but when this is done, the software timer will not actually start until the time at which the scheduler starts.

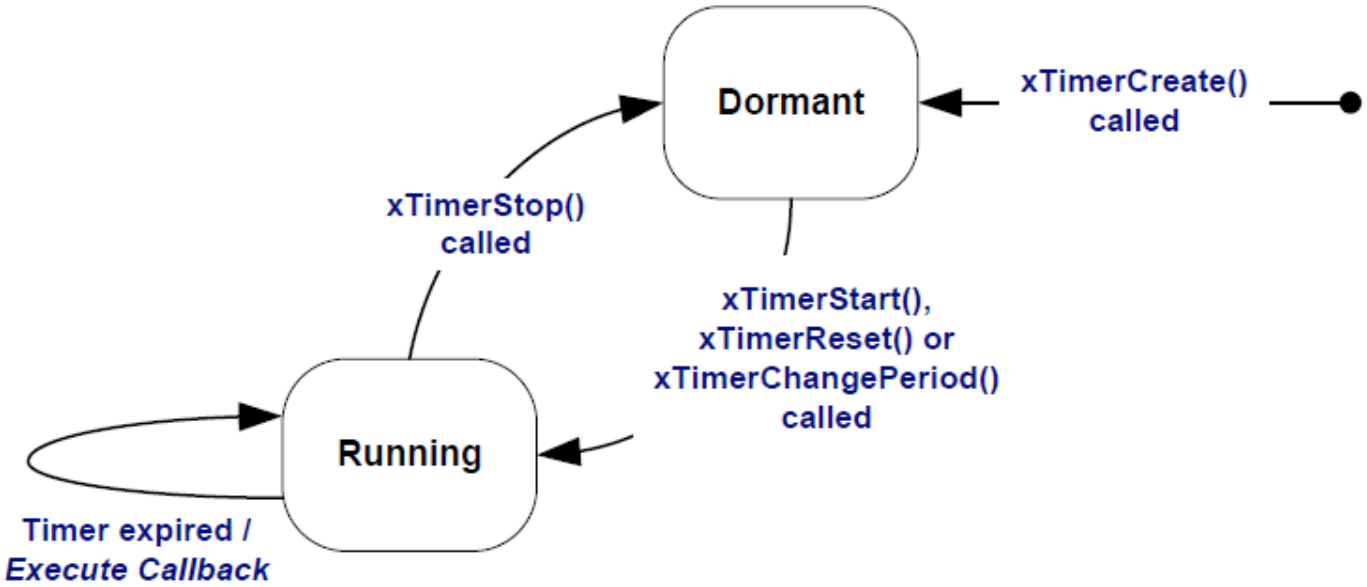
```
TimerHandle_t xTimerStart( TimerHandle_t xTimer, TickType_t xTicksToWait );
```



FreeRTOS Software Timer Start

xTimer is the handle of the software timer being started or restarted. The handle will have been returned from the call to *xTimerCreate()* used to create the software timer.

```
TimerHandle_t xTimerStart( TimerHandle_t xTimer, TickType_t xTicksToWait );
```



FreeRTOS Software Timer Start

xTimerStart() uses the timer command queue to send the *start timer* command to the daemon task.

xTicksToWait specifies the maximum amount of time the calling task should remain in the *Blocked* state to wait for space to become available on the timer command queue, should the queue already be full.

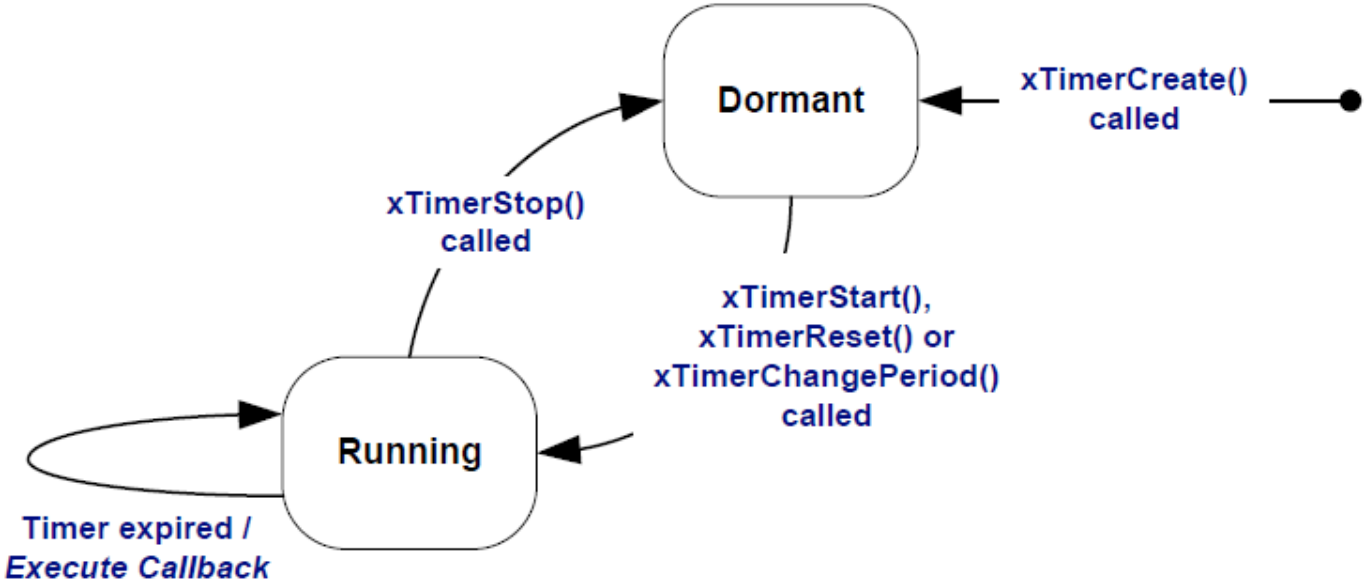
xTimerStart() will return immediately if *xTicksToWait* is zero and the timer command queue is already full.

```
TimerHandle_t xTimerStart( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

FreeRTOS Software Timer Start

xTicksToWait is specified in tick periods, so the absolute time it represents is dependent on the tick frequency. The macro pdMS_TO_TICKS() can be used to convert a time specified in milliseconds into a time specified in ticks.

```
TimerHandle_t xTimerStart( TimerHandle_t xTimer, TickType_t xTicksToWait );
```



FreeRTOS Software Timer Start

If *INCLUDE_vTaskSuspend* is set to 1 in *FreeRTOSConfig.h* then setting *xTicksToWait* to *portMAX_DELAY* will result in the calling task remaining in the *Blocked* state indefinitely to wait for space to become available in the timer command queue.

#define INCLUDE_vTaskSuspend 1

```
TimerHandle_t xTimerStart( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

FreeRTOS Software Timer Start

If *xTimerStart()* is called before the scheduler has been started then the value of *xTicksToWait* is ignored, and *xTimerStart()* behaves as if *xTicksToWait* had been set to zero.

There are two possible return values:

1. *pdPASS* is returned only if the *start timer* command was successfully sent to the timer command queue.

```
TimerHandle_t xTimerStart( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

FreeRTOS Software Timer Start

If the priority of the daemon task is above the priority of the task that called *xTimerStart()*, then the scheduler will ensure the start command is processed before xTimerStart() returns.

This is because the daemon task will pre-empt the task that called xTimerStart() as soon as there is data in the timer command queue.

```
TimerHandle_t xTimerStart( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

FreeRTOS Software Timer Start

If a block time was specified (*xTicksToWait* was not zero), then it is possible the calling task was placed into the *Blocked* state to wait for space to become available in the timer command queue before the function returned, but data was successfully written to the timer command queue before the block time expired.

```
TimerHandle_t xTimerStart( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

FreeRTOS Software Timer Start

2. *pdFALSE* is returned if the *start timer* command could not be written to the timer command queue because the queue was already full.

   If a block time was specified (*xTicksToWait* was not zero) then the calling task will have been placed into the Blocked state to wait for the daemon task to make room in the timer command queue, but the specified block time expired before that happened.

The next example creates and starts a one-shot and auto-load software timer.

FreeRTOS Software Timer Start

```
/* FreeRTOS.org includes. */
#include "FreeRTOS.h“
Example 013
#include "task.h"
#include "timers.h"

/* The periods assigned to the one-shot and auto-reload timers respectively. */
#define mainONE_SHOT_TIMER_PERIOD     ( pdMS_TO_TICKS( 3333UL ) )
#define mainAUTO_RELOAD_TIMER_PERIOD ( pdMS_TO_TICKS( 500UL ) )
```

FreeRTOS Software Timer Start

Listing 75

```
/* The periods assigned to the one-shot and auto-reload timers are 3.333 second and half a
second respectively. */
#define mainONE_SHOT_TIMER_PERIOD     pdMS_TO_TICKS( 3333 )
#define mainAUTO_RELOAD_TIMER_PERIOD  pdMS_TO_TICKS( 500 )

int main( void )
{
TimerHandle_t xAutoReloadTimer, xOneShotTimer;
BaseType_t xTimer1Started, xTimer2Started;

   /* Create the one shot timer, storing the handle to the created timer in xOneShotTimer. */
   xOneShotTimer = xTimerCreate(
                   /* Text name for the software timer - not used by FreeRTOS. */
                   "OneShot",
                   /* The software timer's period in ticks. */
                   mainONE_SHOT_TIMER_PERIOD,
                   /* Setting uxAutoRealod to pdFALSE creates a one-shot software timer. */
                   pdFALSE,
                   /* This example does not use the timer id. */
                   0,
                   /* The callback function to be used by the software timer being created. */
                   prvOneShotTimerCallback );

   /* Create the auto-reload timer, storing the handle to the created timer in xAutoReloadTimer. */
   xAutoReloadTimer = xTimerCreate(
                   /* Text name for the software timer - not used by FreeRTOS. */
                   "AutoReload",
                   /* The software timer's period in ticks. */
                   mainAUTO_RELOAD_TIMER_PERIOD,
                   /* Setting uxAutoRealod to pdTRUE creates an auto-reload timer. */
                   pdTRUE,
                   /* This example does not use the timer id. */
                   0,
                   /* The callback function to be used by the software timer being created. */
                   prvAutoReloadTimerCallback );
```

FreeRTOS Software Timer Start

```
    /* Check the software timers were created. */
    if( ( xOneShotTimer != NULL ) && ( xAutoReloadTimer != NULL ) )
    {
        /* Start the software timers, using a block time of 0 (no block time).  The scheduler has
        not been started yet so any block time specified here would be ignored anyway. */
        xTimer1Started = xTimerStart( xOneShotTimer, 0 );
        xTimer2Started = xTimerStart( xAutoReloadTimer, 0 );

        /* The implementation of xTimerStart() uses the timer command queue, and xTimerStart()
        will fail if the timer command queue gets full.  The timer service task does not get
        created until the scheduler is started, so all commands sent to the command queue will
        stay in the queue until after the scheduler has been started.  Check both calls to
        xTimerStart() passed. */
        if( ( xTimer1Started == pdPASS ) && ( xTimer2Started == pdPASS ) )
        {
            /* Start the scheduler. */
            vTaskStartScheduler();
        }
    }

    /* As always, this line should not be reached. */
    for( ;; );
}
```

Listing 75

The software timers *callback functions* just print a message each time they are called.

FreeRTOS Software Timer Start

Listing 76

```
static void prvOneShotTimerCallback( TimerHandle_t xTimer )
{
TickType_t xTimeNow;

    /* Obtain the current tick count. */
    xTimeNow = xTaskGetTickCount();

    /* Output a string to show the time at which the callback was executed. */
    vPrintStringAndNumber( "One-shot timer callback executing", xTimeNow );

    /* File scope variable. */
    ulCallCount++;
}
```

Listing 77

```
static void prvAutoReloadTimerCallback( TimerHandle_t xTimer )
{
TickType_t xTimeNow;

    /* Obtain the current tick count. */
    xTimeNow = uxTaskGetTickCount();

    /* Output a string to show the time at which the callback was executed. */
    vPrintStringAndNumber( "Auto-reload timer callback executing", xTimeNow );

    ulCallCount++;
}
```

FreeRTOS Software Timer ID

Each software timer has an ID, which is a tag value that can be used by the application. The ID is stored in a void pointer (void *) and can store an integer value directly, point to any other object, or be used as a function pointer.

An initial value is assigned to the ID when the software timer is created. The ID can be updated using the *vTimerSetTimerID()* API function and queried using the *pvTimerGetTimerID()* API function.

```
void vTimerSetTimerID( const TimerHandle_t xTimer, void *pvNewID );

void *pvTimerGetTimerID( TimerHandle_t xTimer );
```

FreeRTOS Software Timer ID

Unlike other software timer API functions, *vTimerSetTimerID()* and *pvTimerGetTimerID()* access the software timer directly and do not send a command to the timer command queue.

xTimer is the handle of the software timer being updated with a new ID value. The handle is returned from the call to *xTimerCreate()* used to create the software timer.

pvNewID is the value to which the software timer's ID will be set.

```
void vTimerSetTimerID( const TimerHandle_t xTimer, void *pvNewID );
```

FreeRTOS Software Timer ID

xTimer is the handle of the software timer being queried. The handle is returned from the call to *xTimerCreate()* used to create the software timer.

Returned value is the ID of the software timer being queried.

```
void *pvTimerGetTimerID( TimerHandle_t xTimer );
```

FreeRTOS Software Timer ID

The same callback function can be assigned to more than one software timer. When that is done, the callback function parameter is used to determine which software timer expired.

The last example uses *two separate* callback functions. One callback function is used by the one-shot timer and the other by the auto-reload timer.

The next example uses the *same* callback function for both software timers.

FreeRTOS Software Timer ID

```
/* Create the one shot timer software timer, storing the handle in xOneShotTimer. */
xOneShotTimer = xTimerCreate( "OneShot",
                              mainONE_SHOT_TIMER_PERIOD,
                              pdFALSE,
                              /* The timer's ID is initialized to 0. */
                              0,
                              /* prvTimerCallback() is used by both timers. */
                              prvTimerCallback );

/* Create the auto-reload software timer, storing the handle in xAutoReloadTimer */
xAutoReloadTimer = xTimerCreate( "AutoReload",
                                 mainAUTO_RELOAD_TIMER_PERIOD,
                                 pdTRUE,
                                 /* The timer's ID is initialized to 0. */
                                 0,
                                 /* prvTimerCallback() is used by both timers. */
                                 prvTimerCallback );
```

Listing 80

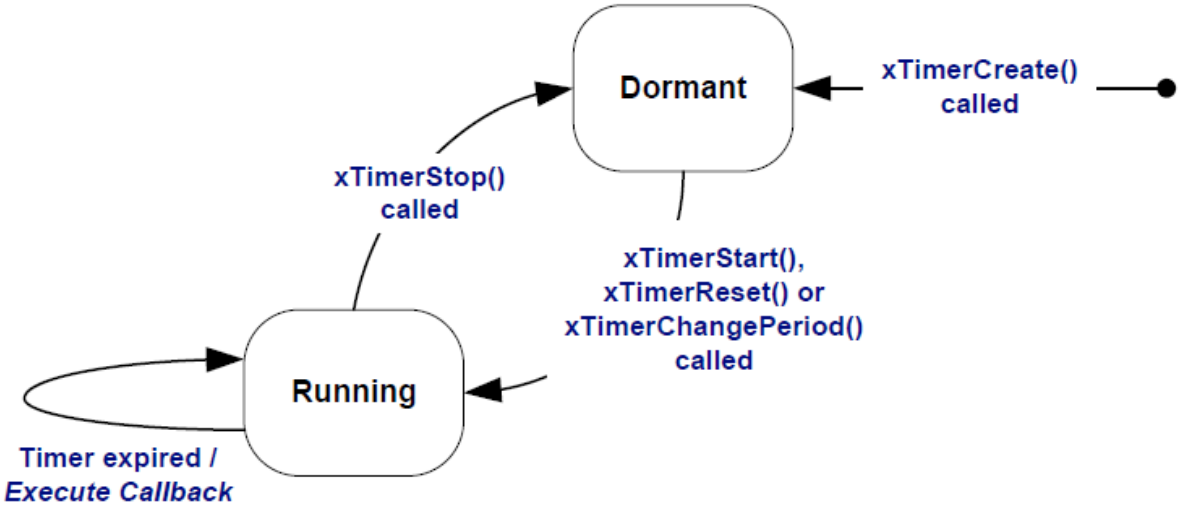
main() in this example is almost identical to last example. The only difference is where the software timers are created. *prvTimerCallback()* is used as the callback function for both timers.

FreeRTOS Software Timer ID

```
static void prvTimerCallback( TimerHandle_t xTimer )
{
TickType_t xTimeNow;
uint32_t ulExecutionCount;

    /* A count of the number of times this software timer has expired is stored in the timer's
    ID.  Obtain the ID, increment it, then save it as the new ID value.  The ID is a void
    pointer, so is cast to a uint32_t. */
    ulExecutionCount = ( uint32_t ) pvTimerGetTimerID( xTimer );
    ulExecutionCount++;
    vTimerSetTimerID( xTimer, ( void * ) ulExecutionCount );

    /* Obtain the current tick count. */
    xTimeNow = xTaskGetTickCount();

    /* The handle of the one-shot timer was stored in xOneShotTimer when the timer was created.
    Compare the handle passed into this function with xOneShotTimer to determine if it was the
    one-shot or auto-reload timer that expired, then output a string to show the time at which
    the callback was executed. */
    if( xTimer == xOneShotTimer )
    {
        vPrintStringAndNumber( "One-shot timer callback executing", xTimeNow );
    }
```

FreeRTOS Software Timer ID

```
    else
    {
        /* xTimer did not equal xOneShotTimer, so it must have been the auto-reload timer that
        expired. */
        vPrintStringAndNumber( "Auto-reload timer callback executing", xTimeNow );

        if( ulExecutionCount == 5 )
        {
            /* Stop the auto-reload timer after it has executed 5 times.  This callback function
            executes in the context of the RTOS daemon task so must not call any functions that
            might place the daemon task into the Blocked state.  Therefore a block time of 0 is
            used. */
            xTimerStop( xTimer, 0 );
        }
    }
}
```



FreeRTOS Software Timer Period

The period of a software timer is changed using the *xTimerChangePeriod()* function. If *xTimerChangePeriod()* is used to change the period of a timer that is already running, then the timer will use the new period value to recalculate its expiry time.

The recalculated expiry time is relative to when *xTimerChangePeriod()* was called, not relative to when the timer was originally started.

FreeRTOS Software Timer Period

If *xTimerChangePeriod()* is used to change the period of a timer that is in the *Dormant* state, then the timer will calculate an expiry time, and transition to the *Running* state.

xTimer is the handle of the software timer being updated with a new period value. The handle will have been returned from the call to *xTimerCreate()* used to create the software timer.

```
BaseType_t xTimerChangePeriod( TimerHandle_t xTimer,
                               TickType_t xNewTimerPeriodInTicks,
                               TickType_t xTicksToWait );
```

FreeRTOS Software Timer Period

xTimerPeriodInTicks is the new period for the software timer, specified in ticks. The *pdMS_TO_TICKS()* macro can be used to convert a time specified in milliseconds into a time specified in ticks.

xTicksToWait specifies the maximum amount of time the calling task should remain in the *Blocked* state to wait for space to become available on the timer command queue, should the queue already be full.

```
BaseType_t xTimerChangePeriod( TimerHandle_t xTimer,
                               TickType_t xNewTimerPeriodInTicks,
                               TickType_t xTicksToWait );
```

FreeRTOS Software Timer Period

xTimerChangePeriod() uses the timer command queue to send the *change period* command to the daemon task.

xTicksToWait xTimerChangePeriod() will return immediately if *xTicksToWait* is zero and the timer command queue is already full. The macro *pdMS_TO_TICKS()* can be used to convert a time specified in milliseconds into a time specified in ticks.

```
BaseType_t xTimerChangePeriod( TimerHandle_t xTimer,
                               TickType_t xNewTimerPeriodInTicks,
                               TickType_t xTicksToWait );
```

FreeRTOS Software Timer Period

If *INCLUDE_vTaskSuspend* is set to 1 in *FreeRTOSConfig.h*, then setting *xTicksToWait* to *portMAX_DELAY* results in the calling task remaining in the *Blocked* state indefinitely to wait for space to become available in the timer command queue.

If *xTimerChangePeriod()* is called before the scheduler has been started, then the value of xTicksToWait is ignored, and xTimerChangePeriod() behaves as if xTicksToWait had been set to zero.

```
BaseType_t xTimerChangePeriod( TimerHandle_t xTimer,
                               TickType_t xNewTimerPeriodInTicks,
                               TickType_t xTicksToWait );
```

FreeRTOS Software Timer Period

There are two possible return values:

1. *pdPASS* is returned only if data was successfully sent to the timer command queue.

   If a block time was specified then it is possible the calling task was placed into the Blocked state to wait for space to become available in the timer command queue before the function returned, but data was successfully written to the timer command queue before the block time expired.

```
BaseType_t xTimerChangePeriod( TimerHandle_t xTimer,
                               TickType_t xNewTimerPeriodInTicks,
                               TickType_t xTicksToWait );
```

FreeRTOS Software Timer Period

There are two possible return values:

2. pdFALSE is returned if the *change period* command could not be written to the timer command queue because the queue was already full.

   If a block time was specified then the calling task is placed into the *Blocked* state to wait for the daemon task to make room in the queue, but the specified block time expired before that happened.

```
BaseType_t xTimerChangePeriod( TimerHandle_t xTimer,
                               TickType_t xNewTimerPeriodInTicks,
                               TickType_t xTicksToWait );
```

FreeRTOS Software Timer Period

The next example shows how the FreeRTOS examples that include self-checking functionality in a software timer callback function use *xTimerChangePeriod()* to increase the rate at which an LED toggles (as an event) if a self-check fails.

The software timer that performs the self-checks is referred to as the *check timer*.

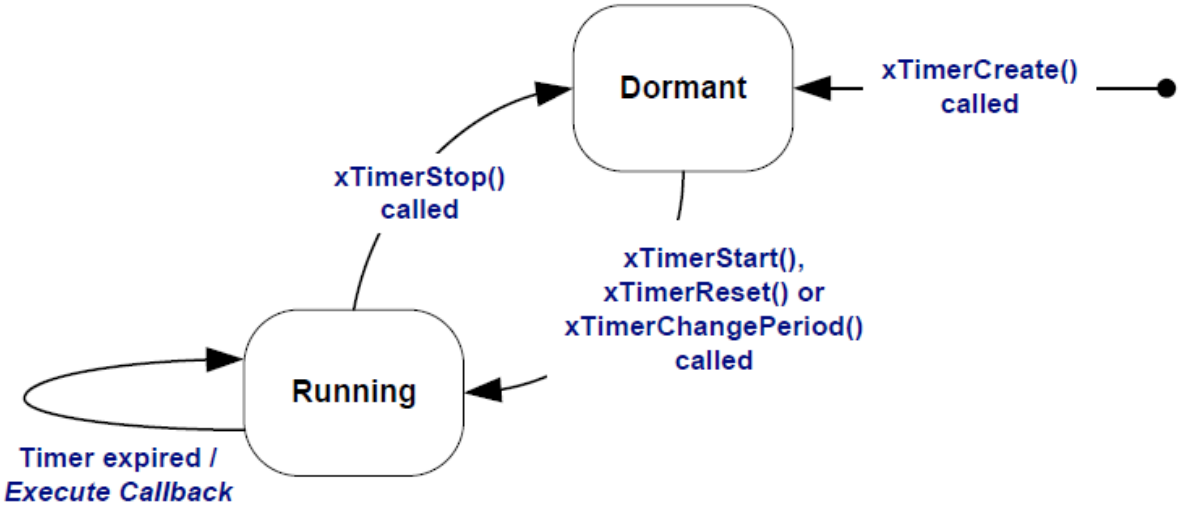
```
BaseType_t xTimerChangePeriod( TimerHandle_t xTimer,
                               TickType_t xNewTimerPeriodInTicks,
                               TickType_t xTicksToWait );
```

FreeRTOS Software Timer Period

```
/* The check timer is created with a period of 3000 milliseconds, resulting in the LED toggling
every 3 seconds.  If the self-checking functionality detects an unexpected state, then the check
timer's period is changed to just 200 milliseconds, resulting in a much faster toggle rate. */
const TickType_t xHealthyTimerPeriod = pdMS_TO_TICKS( 3000 );
const TickType_t xErrorTimerPeriod = pdMS_TO_TICKS( 200 );

/* The callback function used by the check timer. */
static void prvCheckTimerCallbackFunction( TimerHandle_t xTimer )
{
static BaseType_t xErrorDetected = pdFALSE;

    if( xErrorDetected == pdFALSE )
    {
        /* No errors have yet been detected.  Run the self-checking function again.  The
        function asks each task created by the example to report its own status, and also checks
        that all the tasks are actually still running (and so able to report their status
        correctly). */
        if( CheckTasksAreRunningWithoutError() == pdFAIL )
        {
            /* One or more tasks reported an unexpected status.  An error might have occurred.
            Reduce the check timer's period to increase the rate at which this callback function
            executes, and in so doing also increase the rate at which the LED is toggled.  This
            callback function is executing in the context of the RTOS daemon task, so a block
            time of 0 is used to ensure the Daemon task never enters the Blocked state. */
            xTimerChangePeriod( xTimer,             /* The timer being updated. */
                                xErrorTimerPeriod, /* The new period for the timer. */
                                0 );               /* Do not block when sending this command. */
        }
```

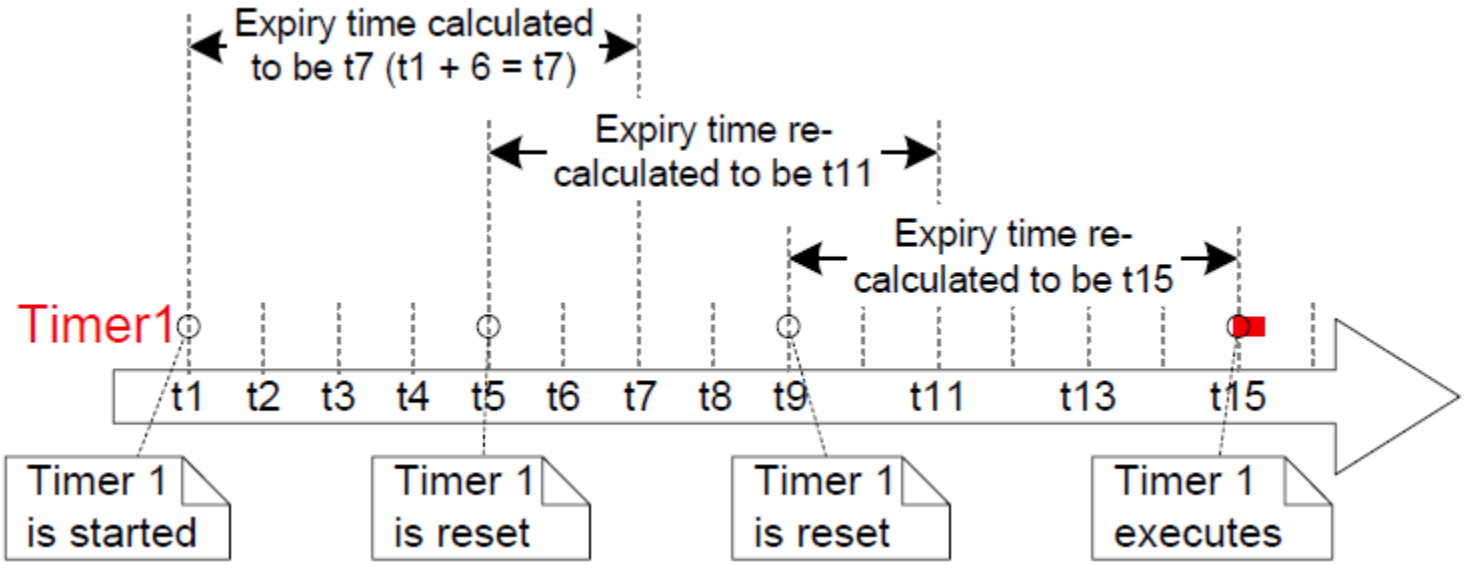
FreeRTOS Software Timer Period

```
        /* Latch that an error has already been detected. */
        xErrorDetected = pdTRUE;
    }

    /* Toggle the LED.  The rate at which the LED toggles will depend on how often this function
    is called, which is determined by the period of the check timer. The timer's period will
    have been reduced from 3000ms to just 200ms if CheckTasksAreRunningWithoutError() has ever
    returned pdFAIL. */
    ToggleLED();
}
```



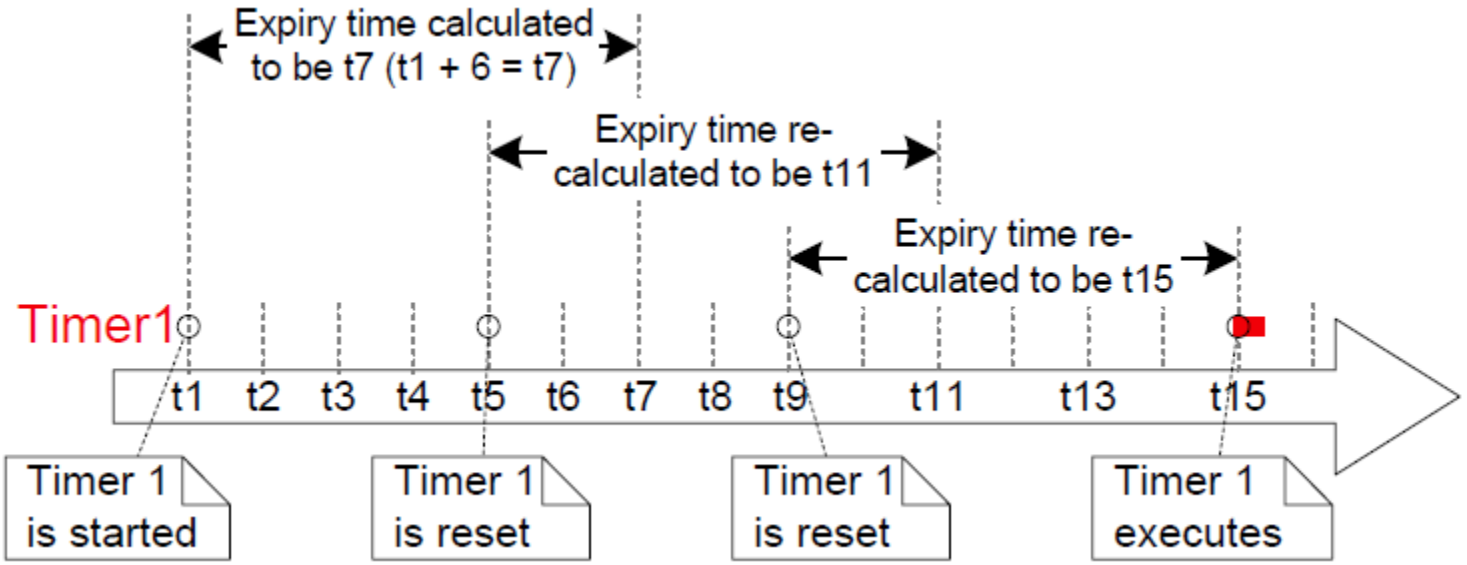
FreeRTOS Resetting a Software Timer

Resetting a software timer means to restart the timer. The expiry time is recalculated to be relative to when the timer was reset, rather than when the timer was originally started.



FreeRTOS Resetting a Software Timer

Here a timer that has a period of 6 is started, then reset twice, before eventually expiring and executing its callback function.



FreeRTOS Resetting a Software Timer

A timer is reset using the *xTimerReset()* API function. *xTimerReset()* can also be used to start a timer that is in the *Dormant* state.

xTimer is the handle of the software timer being reset or started. The handle is returned from the call to *xTimerCreate()* used to create the software timer.

xTicksToWait xTimerChangePeriod() uses the timer command queue to send the *reset* command to the daemon task.

```
BaseType_t xTimerReset( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

FreeRTOS Resetting a Software Timer

xTicksToWait specifies the maximum amount of time the calling task should remain in the *Blocked* state to wait for space to become available on the timer command queue, should the queue already be full.

xTimerReset() will return immediately if xTicksToWait is zero and the timer command queue is already full.

```
BaseType_t xTimerReset( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

FreeRTOS Resetting a Software Timer

If *INCLUDE_vTaskSuspend* is set to 1 in *FreeRTOSConfig.h* then setting *xTicksToWait* to *portMAX_DELAY* results in the calling task remaining in the Blocked state indefinitely to wait for space to become available in the timer command queue.

#define INCLUDE_vTaskSuspend
1

```
BaseType_t xTimerReset( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

FreeRTOS Resetting a Software Timer

There are two possible return values:

1. *pdPASS* is returned only if data was successfully sent to the timer command queue.

   If a block time was specified then it is possible the calling task was placed into the Blocked state to wait for space to become available in the timer command queue before the function returned, but data was successfully written to the timer command queue before the block time expired.

```
BaseType_t xTimerReset( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

FreeRTOS Resetting a Software Timer

There are two possible return values:

2. pdFALSE is returned if the *change period* command could not be written to the timer command queue because the queue was already full.

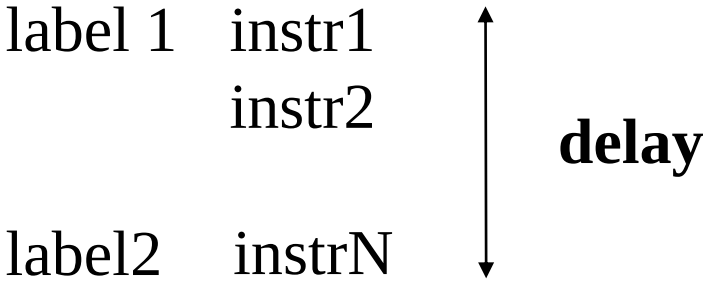
   If a block time was specified then the calling task is placed into the *Blocked* state to wait for the daemon task to make room in the queue, but the specified block time expired before that happened.

```
BaseType_t xTimerReset( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

AXI Timer

Functions of a Typical Timer (1)

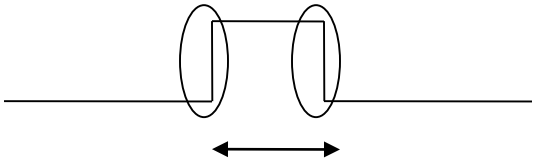
1. **Generating delays - imposing a specific delay between two points in the program**



Functions of a Typical Timer (2)

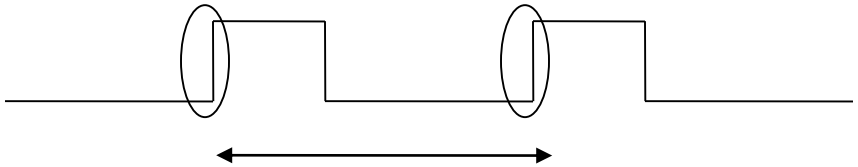
2. Output compare - generating signals with the given timing characteristics

single pulse



pulse width

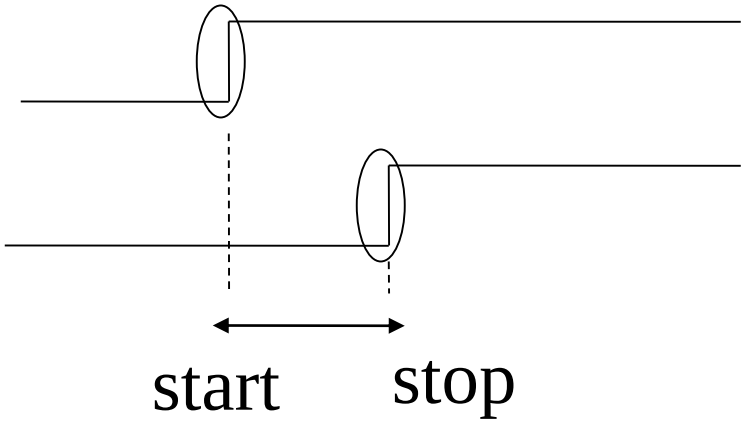
periodical signal

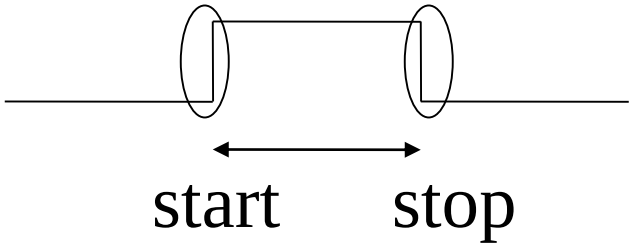


period

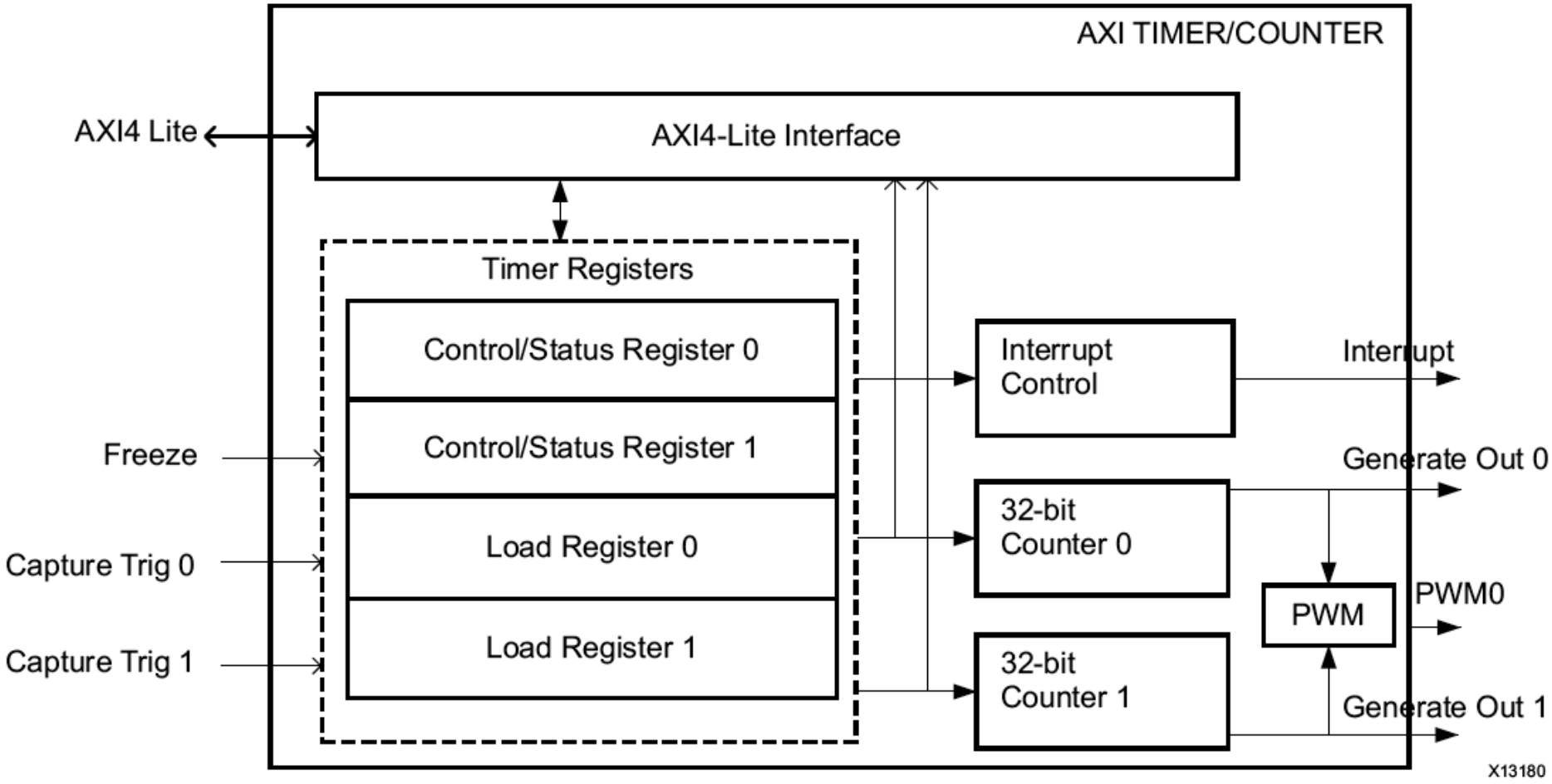
Functions of a Typical Timer (3)

3. Input capture - measuring the time between signal edges





Block Diagram of AXI Timer



Source: LogiCORE IP AXI Timer: Product Guide

AXI Timer: Modes of Operation

- Generate Mode
- Capture Mode
- Pulse Width Modulation Mode
- Cascade Mode

1.#define XTC_CASCADE_MODE_OPTION 0x00000080UL
2.#define XTC_ENABLE_ALL_OPTION 0x00000040UL
3.#define XTC_DOWN_COUNT_OPTION 0x00000020UL
4.#define XTC_CAPTURE_MODE_OPTION 0x00000010UL
5.#define XTC_INT_MODE_OPTION 0x00000008UL
6.#define XTC_AUTO_RELOAD_OPTION 0x00000004UL
7.#define XTC_EXT_COMPARE_OPTION 0x00000002UL

Generate Mode

- Counter when enabled begins to count up or down
- On transition of carry out, the counter
  - stops, or
  - automatically reloads the initial value from the load register, and continues counting
  - if enabled, GenerateOut is driven to 1 for one clock cycle
  - if enabled, the interrupt signal for the timer is driven to 1
- Can be used to
  - Generate repetitive interrupts
  - One-time pulses
  - Periodical signals

Capture Mode

- The counter can be configured as an up or down counter
- The value of the counter is stored in the load register when the external capture signal is asserted
- The TINT flag is also set on detection of the capture event
- The Auto Reload/Hold (ARHT) bit controls whether the capture value is overwritten with a new capture value before the previous TINT flag is cleared

- Can be used to measure
  - Widths of non-periodical signals
  - Periods of periodical signals
  - Intervals between edges of two different signals, etc.

Pulse Width Modulation (PWM) Mode

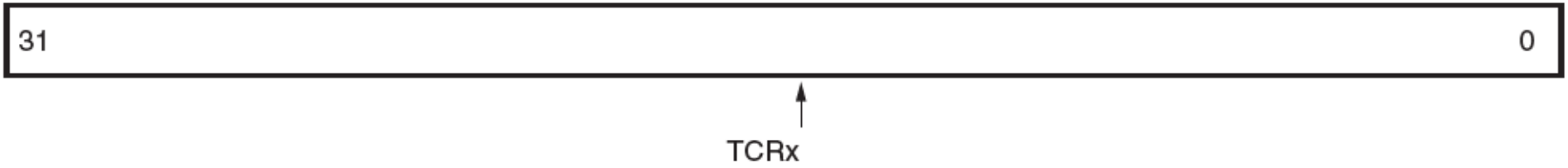
- Two timer/counters are used as a pair to produce an output signal (PWM0) with a specified frequency and duty factor
- Timer 0 sets the period
- Timer 1 sets the high time for the PWM0 output

- Can be used to generate
  - Periodical signals with varying period and duty cycle

Cascade Mode

- Two timer/counters are cascaded to operate as a single 64-bit counter/timer
- The cascaded counter can work in both generate and capture modes
- TCSR0 acts as the control and status register for the cascaded counter. TCSR1 is ignored in this mode.
- Can be used to
  - Generate longer delays
  - Generate signals with larger pulse widths or periods
  - Measure longer time intervals

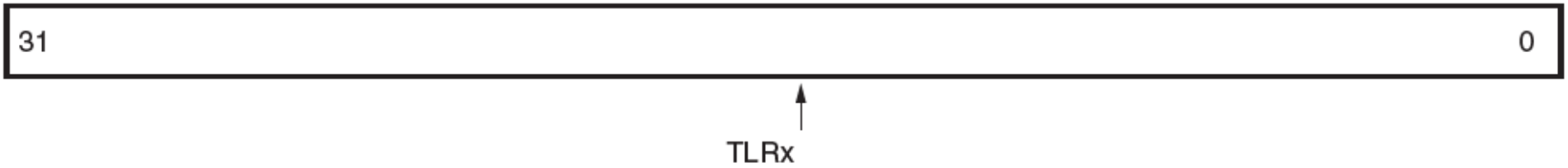
Timer/Counter Register, TCR0, TCR1



Bit	Name	Access Type	Reset Value	Description
31-0	Timer/Counter Register	Read	0x0	Timer/Counter register

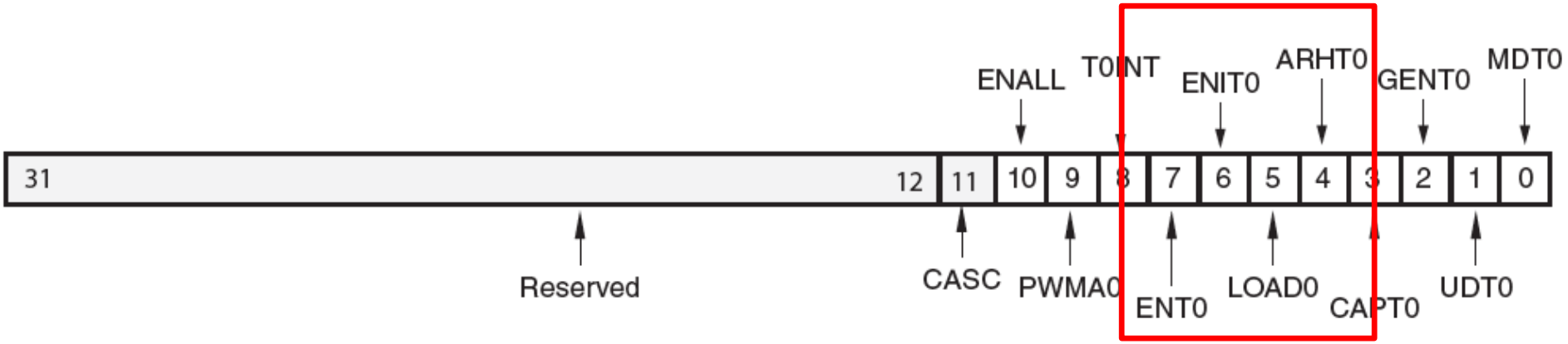
Source: LogiCORE IP AXI Timer: Product Guide

Load Register, TLR0, TLR1



Bit	Name	Access Type	Reset Value	Description
31-0	Timer/Counter Load Register	Read/Write	0x0	Timer/Counter Load register

Control/Status Registers, TCSR0

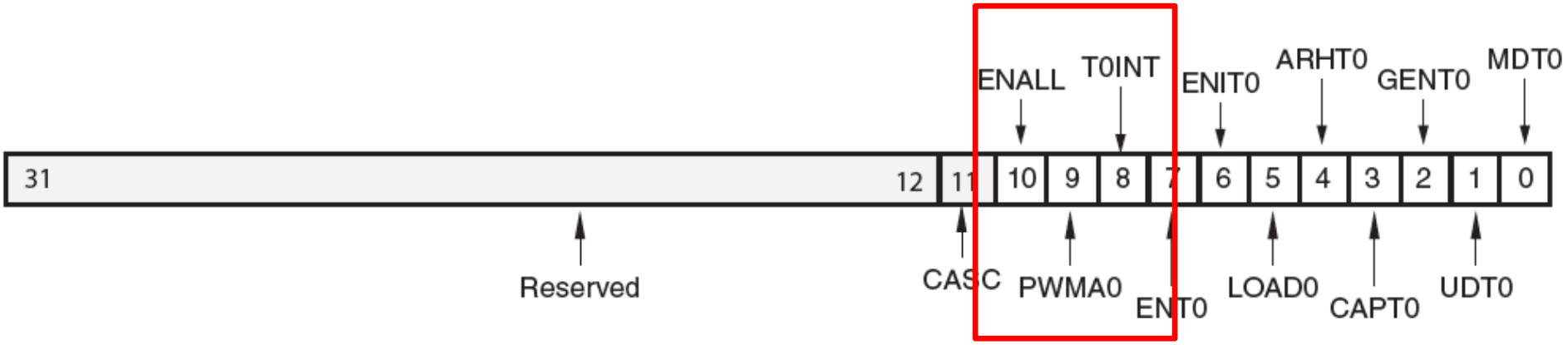


Source: LogiCORE IP AXI Timer: Product Guide

Control/Status Register 0, TCSR0

7	ENT0	Read/Write	0	Enable Timer 0 0 = Disable timer (counter halts) 1 = Enable timer (counter runs)
6	ENIT0	Read/Write	0	Enable Interrupt for Timer 0 Enables the assertion of the interrupt signal for this timer. Has no effect on the interrupt flag (T0INT) in TCSR0. 0 = Disable interrupt signal 1 = Enable interrupt signal
5	LOAD0	Read/Write	0	Load Timer 0 0 = No load 1 = Loads timer with value in TLR0 Setting this bit loads timer/counter register (TCR0) with a specified value in the timer/counter load register (TLR0). This bit prevents the running of the timer/counter; hence, this should be cleared alongside setting Enable Timer/ Counter (ENT0) bit in TCSR0.
4	ARHT0	Read/Write	0	Auto Reload/Hold Timer 0 When the timer is in Generate mode, this bit determines whether the counter reloads the generate value and continues running or holds at the termination value. In Capture mode, this bit determines whether a new capture trigger overwrites the previous captured value or if the previous value is held. 0 = Hold counter or capture value. The TLR must be read before providing the external capture. 1 = Reload generate value or overwrite capture value

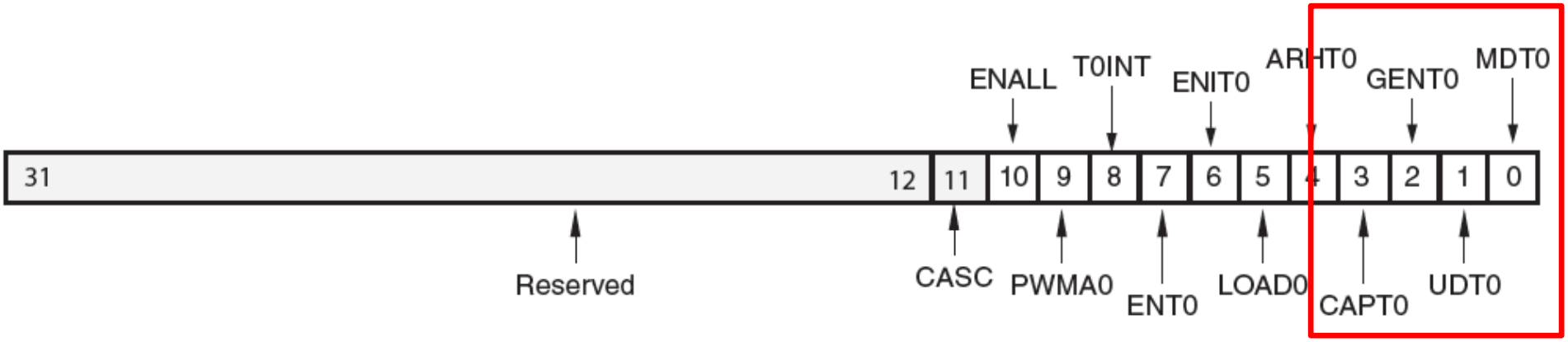
Control/Status Registers, TCSR0



Control/Status Register 0, TCSR0

10	ENALL	Read/Write	0	Enable All Timers 0 = No effect on timers 1 = Enable all timers (counters run) This bit is mirrored in all control/status registers and is used to enable all counters simultaneously. Writing a 1 to this bit sets ENALL, ENT0, and ENT1. Writing a 0 to this register clears ENALL but has no effect on ENT0 and ENT1.
9	PWMA0	Read/Write	0	Enable Pulse Width Modulation for Timer 0 0 = Disable pulse width modulation 1 = Enable pulse width modulation PWM requires using Timer 0 and Timer 1 together as a pair. Timer 0 sets the period of the PWM output, and Timer 1 sets the high time for the PWM output. For PWM mode, MDT0 and MDT1 must be 0 and C_GEN0_ASSERT and C_GEN1_ASSERT must be 1.
8	T0INT	Read/Write	0	Timer 0 Interrupt Indicates that the condition for an interrupt on this timer has occurred. If the timer mode is capture and the timer is enabled, this bit indicates a capture has occurred. If the mode is generate, this bit indicates the counter has rolled over. Must be cleared by writing a 1. *Read:* 0 = No interrupt has occurred 1 = Interrupt has occurred *Write:* 0 = No change in state of T0INT 1 = Clear T0INT (clear to 0)

Control/Status Registers, TCSR0



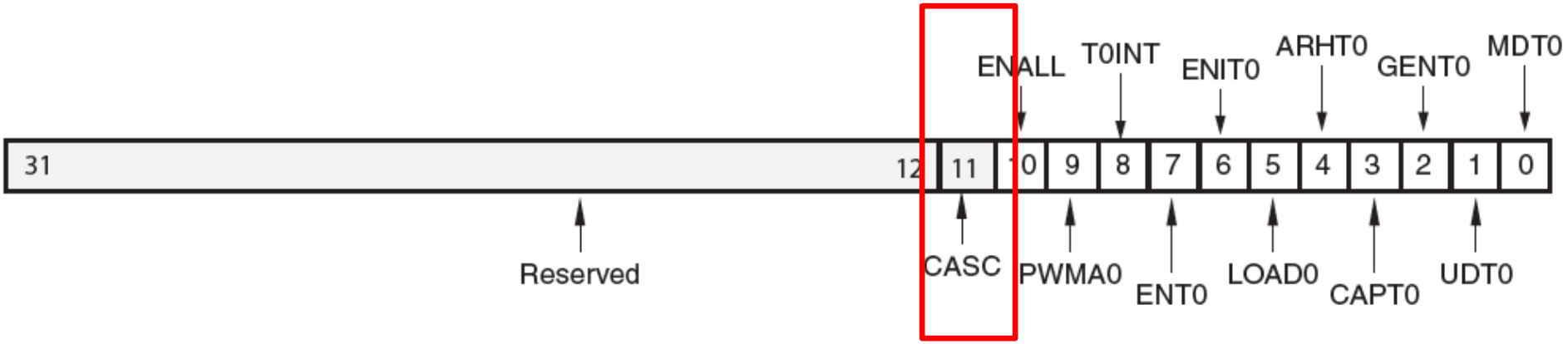
Source: LogiCORE IP AXI Timer: Product Guide

Control/Status Register 0, TCSR0

Bit	Name	Access Type	Reset Value	Description
3	CAPT0	Read/Write	0	Enable External Capture Trigger Timer 0 0 = Disables external capture trigger 1 = Enables external capture trigger
2	GENT0	Read/Write	0	Enable External Generate Signal Timer 0 0 = Disables external generate signal 1 = Enables external generate signal
1	UDT0	Read/Write	0	Up/Down Count Timer 0 0 = Timer functions as up counter 1 = Timer functions as down counter
0	MDT0	Read/Write	0	Timer 0 Mode See Timer Modes. 0 = Timer mode is generate 1 = Timer mode is capture

Source: LogiCORE IP AXI Timer: Product Guide

Control/Status Registers, TCSR0



Source: LogiCORE IP AXI Timer: Product Guide

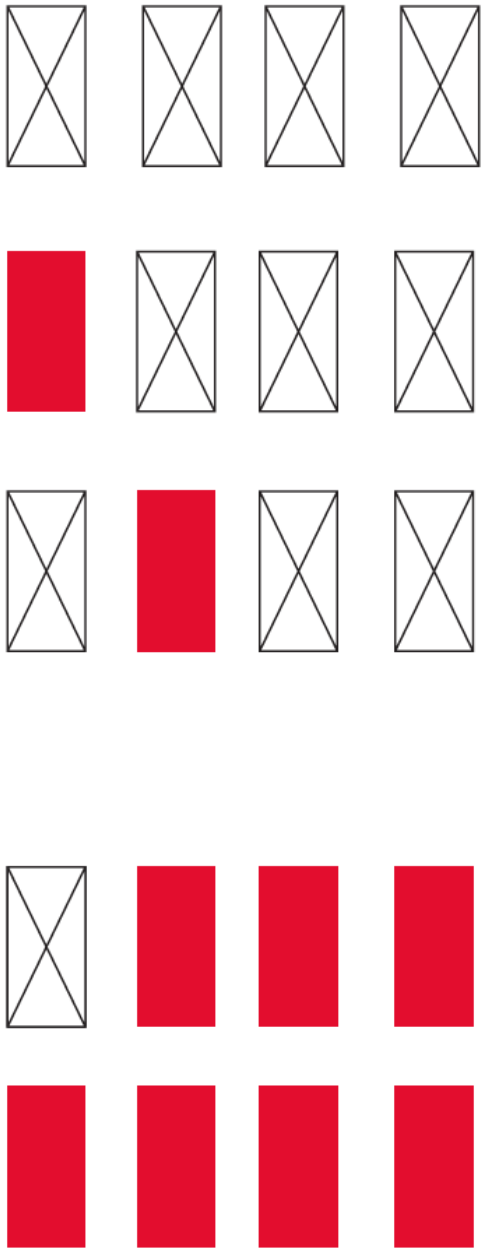
Control/Status Register 0, TCSR0

11	CASC	Read/Write	0	Enable cascade mode of timers 0 = Disable cascaded operation 1 = Enable cascaded operation Cascaded operation requires using Timer 0 and Timer 1 together as a pair. The counting event for the Timer 1 is when the Timer 0 rolls over from all 1s to all 0s or vice-versa when counting down. TLR0 and TLR1 are used for lower 32-bit and higher 32-bit respectively. Similarly, TCR0 contains lower 32-bits for the 64-bit counter and TCR1 contains the higher 32-bits. Only TCSR0 is valid for both the timer/counters in this mode. This CASC bit must be set before enabling the timer/ counter.

Source: LogiCORE IP AXI Timer: Product Guide

Lab 4:

Modifying a Counter Using AXI Timer (every N ms)



C Program (1)

```
#include "xparameters.h"
#include "xgpio.h"
#include "xtmrctr.h"
#include "xscugic.h"
#include "xil_exception.h"
#include "xil_printf.h"

// Parameter definitions
#define INTC_DEVICE_ID          XPAR_PS7_SCUGIC_0_DEVICE_ID
#define TMR_DEVICE_ID           XPAR_TMRCTR_0_DEVICE_ID
#define BTNS_DEVICE_ID          XPAR_AXI_GPIO_0_DEVICE_ID
#define LEDS_DEVICE_ID          XPAR_AXI_GPIO_1_DEVICE_ID
#define INTC_GPIO_INTERRUPT_ID
        XPAR_FABRIC_AXI_GPIO_0_IP2INTC_IRPT_INTR
#define INTC_TMR_INTERRUPT_ID
        XPAR_FABRIC_AXI_TIMER_0_INTERRUPT_INTR

#define BTN_INT                         XGPIO_IR_CH1_MASK
#define TMR_LOAD                        0xF8000000
```

C Program (2)

```
XGpio LEDInst, BTNInst;
XScuGic INTCInst;
XTmrCtr TMRInst;
static int led_data;
static int btn_value;
static int tmr_count;

//----------------------------------------------------
// PROTOTYPE FUNCTIONS
//----------------------------------------------------
static void BTN_Intr_Handler(void *InstancePtr);
static void TMR_Intr_Handler(void *InstancePtr);
static int InterruptSystemSetup(XScuGic *XScuGicInstancePtr);
static int IntcInitFunction(u16 DeviceId, XTmrCtr *TmrInstancePtr, XGpio
*GpioInstancePtr);
```

C Program (3A)

```
void BTN_Intr_Handler(void *InstancePtr)
{
   // Disable GPIO interrupts
   XGpio_InterruptDisable(&BTNInst, BTN_INT);
   // Ignore additional button presses
   if ((XGpio_InterruptGetStatus(&BTNInst) & BTN_INT) !=BTN_INT) {
        return;
   }
   btn_value = XGpio_DiscreteRead(&BTNInst, 1);
   // Increment counter based on button value
   // Reset if center button pressed
   if(btn_value != 8)
        led_data = led_data + btn_value;
   else
        led_data = 0;
   XGpio_DiscreteWrite(&LEDInst, 1, led_data);
   (void) XGpio_InterruptClear(&BTNInst, BTN_INT);
    // Enable GPIO interrupts
   XGpio_InterruptEnable(&BTNInst, BTN_INT);
}
```

C Program (3B)

```
void TMR_Intr_Handler(void *InstancePtr)
{
        if (XTmrCtr_IsExpired(&TMRInst, 0)){
                // Once timer has expired 3 times, stop, increment
counter
                // reset timer and start running again
                if(tmr_count == 3){
                        XTmrCtr_Stop(&TMRInst, 0);
                        tmr_count = 0;
                        led_data++;
                        XGpio_DiscreteWrite(&LEDInst, 1, led_data);
                        XTmrCtr_Reset(&TMRInst, 0);
                        XTmrCtr_Start(&TMRInst, 0);

                }
                else tmr_count++;
        }
}
```

C Program (4A)

```
int main (void)
{
    int status;
    // Initialise LEDs
     status = XGpio_Initialize(&LEDInst, LEDS_DEVICE_ID);
     if(status != XST_SUCCESS) return XST_FAILURE;
     // Initialize Push Buttons
     status = XGpio_Initialize(&BTNInst, BTNS_DEVICE_ID);
     if(status != XST_SUCCESS) return XST_FAILURE;
     // Set LEDs direction to outputs
     XGpio_SetDataDirection(&LEDInst, 1, 0x00);
     // Set all buttons direction to inputs
     XGpio_SetDataDirection(&BTNInst, 1, 0xFF);
```

C Program (4A)

```
//-----------------------------------------------
 // SETUP THE TIMER
 //-----------------------------------------------
 status = XTmrCtr_Initialize(&TMRInst, TMR_DEVICE_ID);
 if(status != XST_SUCCESS)
     return XST_FAILURE;
 XTmrCtr_SetHandler(&TMRInst, TMR_Intr_Handler, &TMRInst);
 XTmrCtr_SetResetValue(&TMRInst, 0, TMR_LOAD);
 XTmrCtr_SetOptions(&TMRInst, 0,
            XTC_INT_MODE_OPTION | XTC_AUTO_RELOAD_OPTION);
```

C Program (4C)

```
    // Initialize interrupt controller
    status = IntcInitFunction(INTC_DEVICE_ID, &BTNInst);
    if(status != XST_SUCCESS) return XST_FAILURE;
    while(1);
    return 0;
}
```

C Program (5)

```
int IntcInitFunction(u16 DeviceId, XTmrCtr *TmrInstancePtr,
                                   XGpio *GpioInstancePtr)
{
   XScuGic_Config *IntcConfig;
   int status;

    // Interrupt controller initialization
    IntcConfig = XScuGic_LookupConfig(DeviceId);
    status = XScuGic_CfgInitialize(&INTCInst, IntcConfig,
                                                IntcConfig->CpuBaseAddress);
    if(status != XST_SUCCESS) return XST_FAILURE;

    // Call to interrupt setup
    status = InterruptSystemSetup(&INTCInst);
    if(status != XST_SUCCESS) return XST_FAILURE;
```

C Program (6A)

```
// Connect GPIO interrupt to handler
status = XScuGic_Connect(&INTCInst, INTC_GPIO_INTERRUPT_ID,
                         (Xil_ExceptionHandler) BTN_Intr_Handler,
                         (void *)GpioInstancePtr);

 if(status != XST_SUCCESS) return XST_FAILURE;

 // Connect timer interrupt to handler
status = XScuGic_Connect(&INTCInst, INTC_TMR_INTERRUPT_ID,
                         (Xil_ExceptionHandler)TMR_Intr_Handler,
                         (void *)TmrInstancePtr);

if(status != XST_SUCCESS) return XST_FAILURE;
```

C Program (6B)

```
    // Enable GPIO interrupts interrupt
     XGpio_InterruptEnable(GpioInstancePtr, 1);
     XGpio_InterruptGlobalEnable(GpioInstancePtr);

     // Enable GPIO and timer interrupts in the controller
     XScuGic_Enable(&INTCInst, INTC_GPIO_INTERRUPT_ID);

     XScuGic_Enable(&INTCInst, INTC_TMR_INTERRUPT_ID);

     return XST_SUCCESS;
}
```

C Program (7)

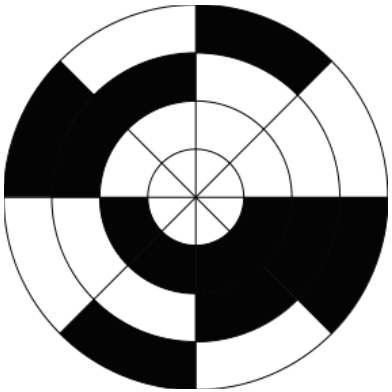
```
int InterruptSystemSetup(XScuGic *XScuGicInstancePtr)
{
   // Enable interrupt
   XGpio_InterruptEnable(&BTNInst, BTN_INT);
   XGpio_InterruptGlobalEnable(&BTNInst);

   Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT,
                                (Xil_ExceptionHandler) XScuGic_InterruptHandler,
                                XScuGicInstancePtr);

   Xil_ExceptionEnable();

   return XST_SUCCESS;

}
```

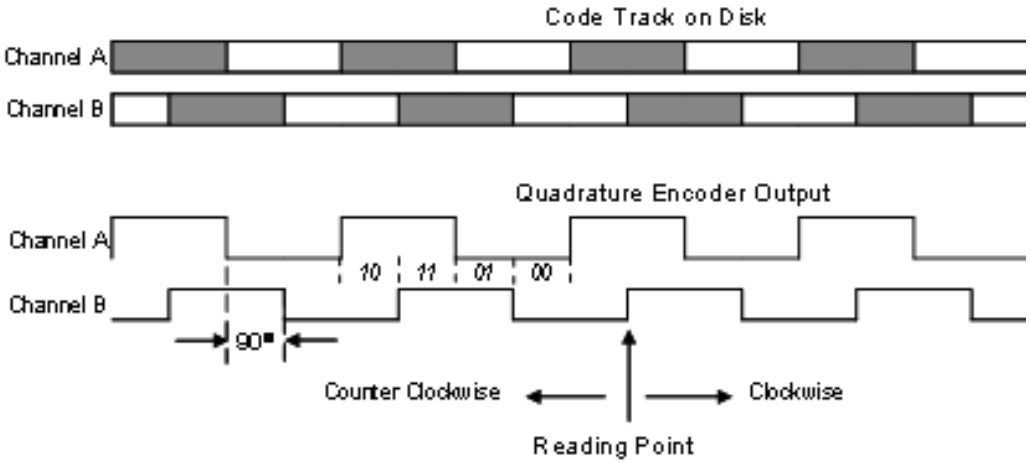
Timer-based Measurement

- Input capture timers can be used to measure quantities from devices such as A/D converters and encoders.
- Example: rotary encoder
  - A rotary encoder is an electro-mechanical device for converting the angular position of a shaft or axle to a digital code.
  - Rotation direction is determined by adding a second sensor placed at a slightly different angle around the shaft from the initial sensor.

Timer-based Measurement

- If A leads B, the disk is rotating in one direction (clockwise in this case). By comparison, if B leads A, then the disk is rotating in the opposite (or counter-clockwise) direction





Example

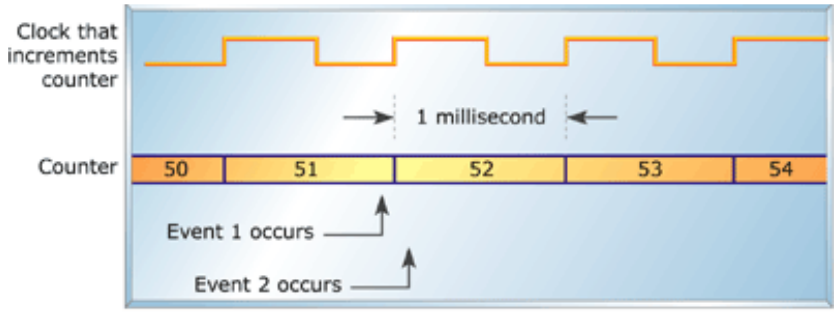
- Suppose that an application uses an embedded processor to control the speed of a DC motor.
  - This motor may rotate at speeds in the range 1 to 3,000 revolutions per minute (RPM).
  - If the encoder produces 100 pulses with each revolution of the shaft.
  - what range of the input capture timer?

Software Timer Management

- Suppose that an application uses an embedded processor to control the speed of a DC motor.
  - This motor may rotate at speeds in the range 1 to 3,000 revolutions per minute (RPM).
  - If the encoder produces 100 pulses with each revolution of the shaft.
  - what range of the input capture timer?

Timer-based Measurement Issues

- Range
  - the range of the 16-bit input capture timer decides all possible motor speed.
- Accuracy
  - the rotational speed of the shaft cannot be calculated with such consistent accuracy across the full range.0
  - As the speed of rotation increases, the difficulty of detecting small changes in speed increases.



Timer-based Measurement Issues

- Sampling Rate
  - Because it is dependent on encoder pulses to trigger input, our example system can measure the motor speed no faster than the encoder pulse rate. As the motor speed decreases, the time between new speed estimates falls. At 3,000 RPM, the sampling rate is 5 kHz; at 2 RPM, it falls to just 0.333Hz.
- Drift
  - When you have two timers operating from different crystal oscillators, you must assume that the values in the timers will diverge over time.